

ETF Trade Optimization Code Design

1. What This Script Is Designed To Do

`optimize_etf_trade.py` is designed to automatically build an ETF create or redemption basket from available dealer inventory.

The script starts from a holdings file that includes current portfolio data, benchmark weight, dealer inventory, and issued amount. It then selects a basket of bonds that tries to:

- Match the requested PNU notional amount.
- Keep the value gap within the configured hard limit.
- Match the benchmark-weighted duration target.
- Keep duration gaps within configured global and bucket-level limits.
- Cover every sector-duration combination.
- Select at least the required number of securities in each sector-duration combination.
- Respect dealer inventory, current holdings for redemption, issued amount caps, and 1,000-share round lots.
- Use as few securities as possible after the more important constraints are addressed.

The code does not use a third-party mathematical optimizer such as Gurobi, CPLEX, `scipy`, or `cvxpy`. Instead, it uses a deterministic greedy/search heuristic that mirrors a manual trader workflow: define targets, find eligible inventory, build a diversified first basket, and then repair the basket one round lot at a time.

2. High-Level Flow

The main workflow is controlled by `main()`. The simplified call flow is:

```
main()
-> read_holdings()
-> benchmark_weighted_duration()
-> duration_bucket_targets()
-> sector_duration_targets()
-> build_candidates()
-> optimize_trade_with_duration_constraints()
-> portfolio_stats()
-> post_trade_portfolio_duration()
-> summarize_by_duration_bucket()
-> summarize_by_sector_duration_combo()
-> write_csv()
```

`main()` acts as the coordinator. It reads inputs, calculates benchmark targets, builds the eligible trading universe, calls the optimizer, calculates result statistics, and writes output files.

3. Configuration And Column Mapping

The top of the script contains a configuration section. This section is intentionally separated from the rest of the code so the script can be moved to another office environment more easily.

The most important object is `COLUMN_MAPPING`.

The left side is the internal variable name used by the script. The right side is the actual column name in the input file.

Example:

```
COLUMN_MAPPING = {  
  "shares": "Shares",  
  "market_value": "Market Value",  
  "duration": "Duration",  
  "dealer_inventory": "Dealer Inventory",  
  "bmk_weight": "Bmk Weight",  
  "issued_amount": "Issued Amount",  
  "sector": "Sector",  
}
```

If an office file uses different column names, the maintainer should usually only change the right side of this mapping.

The rest of the code uses stable internal names such as `shares`, `duration`, or `dealer_inventory`. This avoids scattering office-specific column names throughout the script.

4. Reading And Standardizing The Holdings

The function `read_holdings(path)` loads the input CSV and enriches each row with internal calculation fields.

Before loading data, it calls `validate_required_columns()` to make sure the mapped required columns exist. If a required column is missing, the script fails early with a clear message asking the user to update `COLUMN_MAPPING`.

Numeric fields are converted by `parse_number()`. This handles spreadsheet-style strings such as 1,234,000.00 and converts blanks to 0.0.

For each valid row, `read_holdings()` calculates:

- ``_shares``
- ``_market_value``
- ``_duration``
- ``_dealer_inventory``
- ``_bmk_weight``
- ``_issued_amount``
- ``_price_per_share``
- ``_sector_group``
- ``_duration_bucket``

- ``_bucket_key``

`_price_per_share` is calculated as:

Market Value / Shares

`_sector_group` is calculated by `sector_group()`. The script maps sectors into:

- ``federal``
- ``gov``, which combines provincial and municipal
- ``corporate``, which is the fallback for everything else

`_duration_bucket` is calculated by `duration_bucket()`. The configured buckets are:

- ``0-5``
- ``5-10``
- ``10-14``
- ``14-30``

`_bucket_key` combines sector group and duration bucket. Examples:

- ``federal|0-5``
- ``gov|5-10``
- ``corporate|14-30``

This standardized data structure is what the optimizer uses. The optimizer does not work directly on raw spreadsheet strings.

5. Benchmark Targets

The optimizer uses `Bmk Weight` to define the target portfolio structure.

The global benchmark duration is calculated by `benchmark_weighted_duration()`:

$\text{sum}(\text{Bmk Weight} * \text{Duration}) / \text{sum}(\text{Bmk Weight})$

Two other functions build target values and target durations:

- ``duration_bucket_targets()``
- ``sector_duration_targets()``

`duration_bucket_targets()` calculates how much trade value should be assigned to each duration bucket, and what the benchmark duration target is for that bucket.

`sector_duration_targets()` does the same at the more detailed sector-duration level. For example, it calculates target value and target duration for combinations such as:

- ``corporate|0-5``
- ``corporate|5-10``

- `federal|10-14`
- `gov|14-30`

For example, if a sector-duration combination represents 20% of total benchmark weight, then 20% of the requested PNU trade amount is assigned to that combination.

This design means the target basket is driven by benchmark weight, not by the current market value weight of the portfolio.

6. Eligible Trading Candidates

The function `build_candidates()` creates the list of securities that the optimizer is allowed to use.

For create trades:

maximum trade shares $\leq$ Dealer Inventory

For redemption trades:

maximum trade shares $\leq$ Dealer Inventory
maximum trade shares $\leq$ Current Shares

The redemption current-shares cap ensures post-trade shares cannot become negative.

Both create and redemption trades also respect the issued amount cap:

$\text{abs}(\text{trade shares}) \leq \text{Issued Amount} * 1000 * 0.5$

Every maximum share amount is rounded down to a 1,000-share round lot.

Each candidate contains:

- The original enriched row.
- Maximum number of round lots.
- Market value of one round lot.
- Duration.
- Sector-duration combination.

This candidate list is the tradable universe used by the optimizer.

7. Main Optimizer

The production optimizer is `optimize_trade_with_duration_constraints()`.

Its workflow is:

1. Split candidates into sector-duration combinations.
2. Check that every required combination has enough eligible securities.

3. Optimize each sector-duration combination locally.
4. Merge all local baskets into one total trade basket.
5. Repair duration gaps.
6. Repair value gap, duration gap, and minimum-security violations using a priority score.

The older functions `optimize_trade()` and `seed_bucket_coverage()` remain in the file as legacy helpers, but the current production path is `optimize_trade_with_duration_constraints()`.

8. Combo-Level Optimization

Each sector-duration combination is optimized by `optimize_duration_bucket()`.

Despite the function name, in the current workflow it is normally called for one sector-duration combination, such as:

```
corporate|5-10
```

The function tries to select at least the required number of securities, currently usually two, while matching:

- The combination's target value.
- The combination's target duration.
- Each security's maximum lot capacity.
- 1,000-share round lots.

It searches combinations of one or two securities. For two securities, it estimates the mix that would best hit the target duration, then searches nearby round-lot counts.

This local search helps the first basket already look like the benchmark at the sector-duration level before the global repair stage begins.

9. Merging Local Baskets

After each sector-duration combination has been optimized, `merge_trades()` combines all local baskets into one total trade basket.

The internal trade basket stores absolute share quantities:

```
{
  row_id: {
    "row": row,
    "shares_abs": absolute_trade_shares
  }
}
```

The sign is added later:

- Create trades output positive trade shares.
- Redemption trades output negative trade shares.

10. Repair Passes

The optimizer uses two repair passes after the first basket is built.

Repair Pass 1: Duration Repair

The first repair pass focuses on duration.

If a duration bucket has a gap outside the configured tolerance, the optimizer adds one round lot at a time within that bucket.

If bucket-level duration is acceptable but global duration is outside the configured tolerance, the optimizer can add from the full candidate list.

The helper functions used here are:

- ``score_after_add_in_duration_bucket()``
- ``score_after_add()``

These functions ask a simple question:

If we add one more 1,000-share lot of this security, does the basket improve?

Repair Pass 2: Hard-Constraint Repair

The second repair pass can either add or remove one round lot.

It uses `constraint_score()` to evaluate whether a trial basket is better than the current basket.

Removing is only allowed if it does not violate the minimum-security requirement for a sector-duration combination.

Adding is only allowed if the security still has unused capacity from dealer inventory, current shares for redemption, and issued amount cap.

11. Constraint Priority

`constraint_score()` is one of the most important functions in the file.

It returns a tuple. Python compares tuples from left to right, so this tuple becomes the business priority order.

The priority order is:

1. Value gap violation beyond the hard CAD limit.
2. Global duration gap violation beyond the configured global limit.
3. Duration-bucket gap violation beyond the configured bucket limit.
4. Minimum-security violation by sector-duration combination.
5. Absolute value gap.

6. Absolute global duration gap.
7. Maximum duration-bucket gap.
8. Number of selected securities.

This means the script first tries to satisfy hard constraints, then improves the quality of the basket, and only then tries to reduce the number of securities.

The current business constraints include:

- Value gap must be within CAD 300 if feasible.
- Global duration gap should be within 0.1.
- Each duration bucket gap should be within 0.2.
- Every sector-duration combination should have at least the configured minimum number of securities.
- Trade shares must be in 1,000-share round lots.
- Trade size must not exceed dealer inventory.
- Redemption trade size must not exceed current shares.
- Trade size must not exceed 50% of issued amount times 1,000.

If the constraints cannot all be satisfied, the optimizer still returns the best basket found. The output summary reports which constraints passed and which failed.

12. Measurement And Summary Functions

Several functions do not make decisions. They measure the basket so the optimizer and output reports can evaluate it.

`portfolio_stats()` calculates:

- Total trade market value.
- Trade weighted duration.

`post_trade_portfolio_duration()` calculates the portfolio duration after applying the create or redemption trade.

`duration_bucket_gaps()` calculates actual duration minus target duration for each duration bucket.

`sector_duration_gaps()` calculates actual duration minus target duration for each sector-duration combination.

`combo_security_counts()` counts how many securities were selected in each sector-duration combination.

The output summary functions are:

- ``summarize_by_duration_bucket()``
- ``summarize_by_sector_duration_combo()``

These create the CSV reports that show value, duration, target duration, and duration gap by bucket or combination.

13. Output Files

The script writes a trade list and summary files.

The trade list shows the selected securities, trade shares, trade market value, dealer inventory, current shares, and post-trade shares.

The summary files show whether the basket matches benchmark targets at different levels:

- Full trade basket.
- Duration bucket.
- Sector-duration combination.

The JSON summary includes:

- Benchmark duration.
- Trade duration.
- Post-trade portfolio duration.
- Trade value.
- Target value.
- Value gap.
- Global duration gap.
- Bucket duration gaps.
- Sector-duration combination gaps.
- Constraint pass or fail status.

14. Plain-English Summary

The script automates a manual ETF basket selection process.

First, it uses benchmark weight to define what the trade basket should look like by duration and sector. Then it filters the universe down to bonds that can actually be traded using dealer inventory, current holdings for redemption, issued amount caps, and round-lot rules.

Next, it builds a starting basket by making sure every sector-duration combination is represented. Each combination is optimized locally so the basket is not only diversified, but also close to the benchmark target for that part of the portfolio.

Finally, the script repairs the combined basket one 1,000-share lot at a time. It prioritizes the hard value gap, then duration constraints, then coverage rules, and only after that tries to reduce the number of securities.

In short, the design is not a black-box mathematical optimizer. It is a transparent, maintainable, rule-based optimizer that follows the same logic a trader might use manually, but applies it consistently and automatically.